LAPORAN PRAKTIKUM W6:
POLYMORPHIC

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:
Rainissa Azizah (251511059)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

Studi Kasus 1: Another Type of Employee

Pada studi kasus ini, ditambahkan satu jenis pegawai baru, yaitu *Commission*, yang merupakan turunan dari *class Hourly*. Berikut kode dari *class Commission*:

```
public class Commission extends Hourly {
    private double totalSales;
    private double commissionRate;

    public Commission (String eName, String eAddress, String ePhone, String
socSecNumber, double rate, double commRate) {
        super (eName, eAddress, ePhone, socSecNumber, rate);
        commissionRate = commRate;
        totalSales = 0;
    }
    public void addSales (double totalSales) {
        this.totalSales += totalSales;
    }
    public double pay () {
        double payment = super.pay() + (totalSales * commissionRate);
        totalSales = 0;
        return payment;
    }
    public String toString () {
        String result = super.toString();
        result += "\nTotal sales: " + totalSales;
        return result;
    }
}
```

Class Commission mewarisi sifat dari *class Hourly* dengan menggunakan kata kunci *extends* sehingga seluruh data dan *method* yang ada di *Hourly* otomatis ikut dimiliki. Terdapat dua variabel baru yang ditambahkan, yaitu totalSales untuk menyimpan total penjualan dan commissionRate untuk menyimpan persentase komisi dalam bentuk desimal. *Constructor*-nya dibuat untuk menerima enam parameter. Lima parameter pertama dikirim ke *constructor Hourly* lewat super, sedangkan parameter keenam disimpan sebagai commissionRate. Total penjualan selalu dimulai dari nol karena pegawai yang baru terdaftar belum memiliki penjualan.

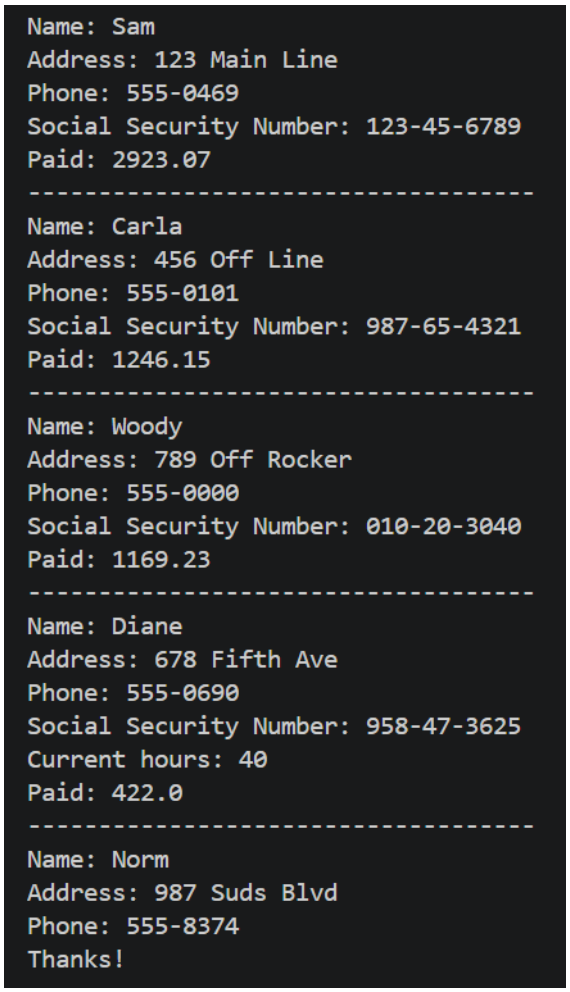
Selain itu, *method* addSales menggunakan operator += sehingga nilai yang diberikan ditambahkan ke total penjualan, bukan menggantikan nilai sebelumnya. Karena nama parameter sama dengan nama variabel milik *class*, kata *this* dipakai untuk membedakan keduanya. Sementara itu, untuk perhitungannya, *method pay* memanggil *pay* milik *Hourly* untuk mendapatkan gaji dari jam kerja, lalu menambahkan hasil kali totalSales dengan commissionRate. Total penjualan dikembalikan ke nol sebelum return karena penjualan tersebut sudah dibayarkan. *Method* toString memanggil toString milik *Hourly*, lalu menambahkan baris total penjualan di bagian akhir.

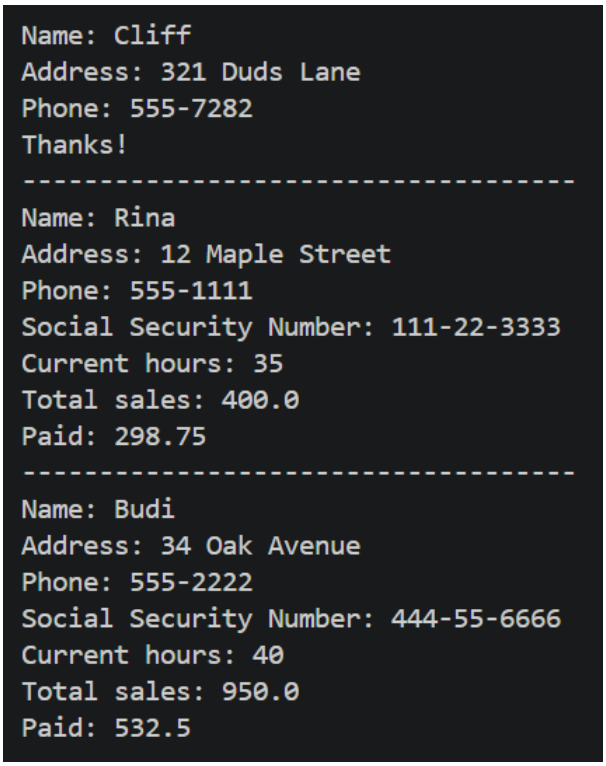
Selanjutnya, Staff.java diubah pada tiga bagian, yaitu:

```
staffList = new StaffMember[8];
...
staffList[6] = new Commission ("Rina", "12 Maple Street", "555-1111", "111-22-3333", 6.25, 0.20);
staffList[7] = new Commission ("Budi", "34 Oak Avenue", "555-2222", "444-55-6666", 9.75, 0.15);
...
((Commission)staffList[6]).addHours (35);
((Commission)staffList[6]).addSales (400);
((Commission)staffList[7]).addHours (40);
((Commission)staffList[7]).addSales (950);
```

Ukuran array perlu diperbesar dari 6 menjadi 8 untuk menampung dua pegawai baru. Pegawai pertama mempunyai tarif $6.25/jam dengan komisi 20%, sedangkan pegawai kedua memunyai tarif $9.75/jam dengan komisi 15%. Data jam kerja dan penjualan diisi lewat addHours dan addSales. Karena array tersebut bertipe StaffMember dan *class* tersebut tidak memiliki kedua method itu, diperlukan *casting* ke *Commission* agar *method* dapat dipanggil. *Method payday* tidak perlu diubah karena proses perulangannya memakai staffList.length.

Berikut adalah output dari Studi Kasus 1:





Studi Kasus 2: Painting Shapes

Pada studi kasus ini, dibuat hierarki bentuk dengan class abstract Shape sebagai induk. Berikut adalah kode dari class Shape:

```
abstract public class Shape {
    protected String shapeName;

    public Shape (String name) {
        shapeName = name;
    }
    public abstract double area();

    public String toString() {
```

```
        return shapeName;
    }
}
```

Class Shape dibuat bersifat *abstract* supaya objeknya tidak dapat dibuat secara langsung. Di dalam *class Shape*, terdapat variabel shapeName yang menyimpan nama bentuk dan diisi lewat *constructor*. Selain itu, *method* area juga dibuat bersifat *abstract* karena hanya memiliki nama dan tanpa isi jadi setiap class turunannya wajib menuliskan rumus luasnya sendiri. Terakhir, *method* toString mengembalikan nama dari bentuk tersebut.

Class Rectangle dan *Cylinder* dibuat dengan pola yang sama seperti *class Sphere* yang sudah tersedia di modul. Berikut adalah kode *Rectangle* dan *Cylinder*:

a. Rectangle

```
public class Rectangle extends Shape {
    private double length;
    private double width;

    public Rectangle (double l, double w) {
        super("Rectangle");
        length = l;
        width = w;
    }
    public double area() {
        return length * width;
    }
    public String toString() {
        return super.toString() + " of length " + length + " and width " +
width;
    }
}
```

b. Cylinder

```
public class Cylinder extends Shape {
    private double radius;
    private double height;

    public Cylinder (double r, double h) {
        super("Cylinder");
        radius = r;
```

```
        height = h;
    }
    public double area() {
        return Math.PI * radius * radius * height;
    }
    public String toString() {
        return super.toString() + " of radius " + radius + " and height " +
 height;
    }
 }
```

Pada kedua *class* tersebut, nama bentuk dikirim ke *constructor Shape* melalui super, lalu data ukurannya disimpan. Untuk perhitungannya, luas *Rectangle* dihitung dari panjang kali lebar. Luas *Cylinder* mengikuti rumus pada modul, yaitu PI dikali radius kuadrat dikali tinggi. *Method* toString memanggil toString milik *Shape* lalu menambahkan keterangan ukuran.

Pada Paint.java, method amount dilengkapi dengan satu baris return.

```
public double amount (Shape s) {
    System.out.println ("Computing amount for " + s);
    return s.area() / coverage;
}
```

Jumlah cat dihitung dengan cara membagi luas bentuk dengan nilai *coverage*. Karena parameter yang digunakan bertipe *Shape* sehingga method ini dapat menerima objek apa saja, baik Rectangle, Sphere, maupun Cylinder. Dengan adanya sifat *polymorphic* tersebut, pemanggilan s.area() otomatis memilih rumus yang sesuai dengan jenis objek yang sedang diproses. Baris print dibiarkan sesuai perintah dokumen.

Pada PaintThings.java, ditambahkan pembuatan objek dan pemanggilan method amount.

```
deck = new Rectangle (20, 35);
bigBall = new Sphere (15);
tank = new Cylinder (10, 30);
deckAmt = paint.amount (deck);
ballAmt = paint.amount (bigBall);
tankAmt = paint.amount (tank);
```

Objek deck didefinisikan sebagai persegi panjang dengan ukuran 20 x 35, bigBall berupa bola dengan radius 15, dan tank berupa tabung dengan radius 10 dan tinggi 30. Selaint itu, bagian pemanggilan amount harus diletakkan sebelum baris yang mencetak hasil karena variabel tidak boleh dipakai sebelum diberi nilai.

Dengan coverage 350, hasilnya adalah 2 galon untuk deck, 8.1 galon untuk bigBall, dan 26.9 galon untuk tank.

```
Computing amount for Rectangle of length 20.0 and width 35.0
Computing amount for Sphere of radius 15.0
Computing amount for Cylinder of radius 10.0 and height 30.0

Number of gallons of paint needed...
Deck 2
Big Ball 8.1
Tank 26.9
```

Studi Kasus 3: Polymorphic Sorting

Pada studi kasus ini, *class Sorting* digunakan untuk mengurutkan data berupa angka, kata, dan sales. Di file Numbers.java awalnya tidak dapat di-*compile* karena menggunakan array bertipe int, sedangkan *Sorting* hanya menerima array berisi objek *Comparable*. Tipe int adalah data primitif jadi tidak memiliki *method* compareTo. Perbaikannya dilakukan pada dua baris.

```
Integer[] intList;
...
intList = new Integer[size];
```

Integer adalah versi objek dari int dan sudah Comparable. Baris lain tidak perlu diubah karena autoboxing otomatis mengubah int hasil scan.nextInt() menjadi Integer.

Program Strings.java dibuat dengan menyalin dan mengedit file Numbers.java.

```
import java.util.Scanner;

public class Strings {
    public static void main (String[] args) {
        String[] stringList;
        int size;

        Scanner scan = new Scanner(System.in);

        System.out.print ("\nHow many strings do you want to sort? ");
        size = scan.nextInt();
        stringList = new String[size];

        System.out.print("\nEnter the strings...");
```

```
        for (int i = 0; i < size; i++)
            stringList[i] = scan.next();
        Sorting.insertionSort(stringList);

        System.out.print ("\nYour strings in sorted order...");
        for (int i = 0; i < size; i++)
            System.out.print (stringList[i] + " ");
        System.out.println ();
    }
}
```

Bentuk array diganti menjadi String[] dan tidak memerlukan autoboxing karena String sudah berupa objek dan sudah *Comparable*. Untuk proses bacanya, isi array menggunakan scan.next() yang membaca satu kata, sedangkan jumlah data tetap dibaca dengan nextInt().

Selanjutnya, *method* insertionSort di Sorting.java diubah menjadi urutan menurun (*descending*). Penyesuaian ini dilakukan pada bagian *looping:*

```
while (position > 0 && key.compareTo(list[position-1]) > 0)
```

Tanda < diganti menjadi >. Elemen yang sedang disisipkan (*key*) digeser ke kiri selama nilainya lebih besar dari elemen di sebelah kirinya, sehingga hasil akhirnya terurut dari besar ke kecil. Pada Numbers.java dan Strings.java, pemanggilan fungsinya di-*update* menjadi Sorting.insertionSort.

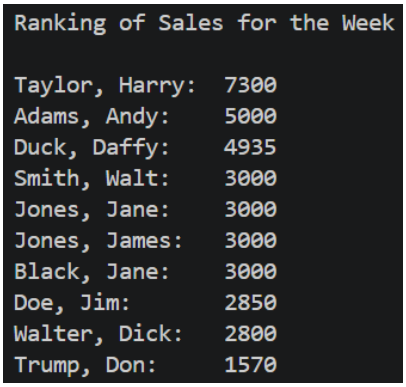
Pada Salesperson.java, method compareTo dilengkapi.

```
public int compareTo(Object other) {
    int result;
    Salesperson s = (Salesperson) other;

    if (totalSales < s.getSales())
        result = -1;
    else if (totalSales > s.getSales())
        result = 1;
    else {
        result = lastName.compareTo(s.getLastName());
        if (result == 0)
            result = firstName.compareTo(s.getFirstName());
    }
    return result;
}
```

Karena parameter other bertipe Object, diperlukan proses *casting* ke Salesperson agar *method getter* dapat dipanggil. Perbandingan utamanya berdasarkan total penjualan, yaitu nilai negatif dikembalikan jika penjualan lebih sedikit dan nilai positif jika lebih banyak. Jika penjualan sama, perbandingan akan dilanjut dengan melihat nama belakang berurutan secara abjad menggunakan *method* compareTo milik String. Jika nama belakangnya pun masih sama, barulah nama depan yang akan digunakan untuk pembandingannya.

Hasil WeeklySales menampilkan daftar dari penjualan terbanyak ke paling sedikit, dengan orang yang penjualannya sama muncul dalam urutan abjad terbalik.



Lesson Learn

Ketiga studi kasus tersebut menunjukkan bagaiman *polymorphism* memungkinkan satu pemanggilan *method* untuk menghasilkan perilaku yang berbeda sesuai jenis objeknya. Hal ini terlihat pada *pay* di array StaffMember, area di method amount, dan compareTo di *class Sorting*. Pemanggilan yang sama tidak perlu diubah ketika jenis objek bertambah, misalnya saat *Commission* ditambahkan. Selain itu, *casting* hanya diperlukan ketika *method* milik *class* turunan dipanggil lewat tipe induk, seperti addSales pada array StaffMember, sedangkan objek turunan dapat langsung dipakai di tempat yang meminta induknya.

Di sisi lain, ditemukan juga kesalahan ketik pada kode salinan, seperti payRate + hoursWorked di *Hourly* dan list.length-1 di selectionSort. Kesalahan seperti ini memnag tidak menimbulkan pesan error sehingga hanya ketahuan dengan membandingkan hasil program dengan perhitungan manual. Oleh karena itu, hasil program perlu selalu diperiksa dan tidak cukup hanya berhenti pada tahap berhasil di-*compile*.

Link Repo: https://github.com/rainissa/PBO-Praktik.git